Learning an identity mapping through three hidden units

ACML Assignment 1 | Manual backpropagation | 8-3-8 autoencoder

Authors: Thai Binh Nguye and Ege Deniz

1. Task and reproducibility

We reconstruct all eight one-hot vectors through an 8-3-8 network [1]. The input matrix is X = I8, with one example per row; targets equal inputs. Both learned layers use sigmoid activations and biases. The model has 24 + 3 + 24 + 8 = 59 trainable parameters. NumPy supplies array operations; forward propagation, backpropagation and full-batch gradient descent are implemented manually.

To run, install NumPy, Matplotlib and a Jupyter environment (for example, `python -m pip install numpy matplotlib jupyterlab`). Open `lab1.ipynb`, select that Python environment, restart the kernel and run all cells in order. No external dataset is needed. The notebook runs 80 experimental trials and writes CSV tables, figures and `baseline_model.npz` under `results/` relative to its working directory; use the Lab1 folder as that directory. Rerunning overwrites these result files. Runtime depends on the machine.

2. Forward pass and manual backpropagation

With m examples, X is (m,8), W1 is (8,3), b1 is (3,), W2 is (3,8), and b2 is (8,). The code uses row examples and separate biases, equivalent to the lecture's column-vector and augmented-bias notation [3, p. 10].

```
sigmoid(z) = 1 / (1 + exp(-z))
H = sigmoid(X @ W1 + b1)
Y_pred = sigmoid(H @ W2 + b2)
L = sum((Y_pred - Y)**2) / (2*m)
```

The loss sums over output units and averages over examples. The half-squared-error objective and chain rule follow the ANN lecture [3, pp. 15-21]. Since sigmoid'(z) = a(1-a):

```
D2 = (Y_pred-Y) * Y_pred * (1-Y_pred) / m
D1 = (D2 @ W2.T) * H * (1-H)
dW2 = H.T @ D2          db2 = sum(D2, axis=0)
dW1 = X.T @ D1          db1 = sum(D1, axis=0)
parameter = parameter - learning_rate * gradient
```

D2 measures the loss derivative at output preactivations. Multiplying by W2.T propagates it to the hidden layer; H(1-H) accounts for its sigmoid derivative. Weight gradients combine incoming activations with deltas. Bias gradients sum deltas over examples because each bias enters its preactivation with derivative 1. The factor 1/m is applied once in D2. All gradients use the same parameters before any update, as required by simultaneous gradient descent [2, pp. 26-27; 3, pp. 16, 21].

We use small random weights: uniform [-0.1s, 0.1s], where s is an initialization scale, and zero initial biases. Random weights break hidden-unit symmetry; the lecture illustrates this principle in its feature-detector example [3, p. 58]. The distribution, bounds, zero biases and seeds are our choices, not prescribed lecture settings. No Xavier initialization, momentum or regularization is used.

3. Verification and learning experiments

As an additional debugging step from Backprop Help [4, section 2.3], we compare all 59 derivatives with central finite differences (epsilon = 1e-5) at the baseline initialization, seed 0, on examples 1, 3 and 6. Maximum absolute discrepancies are W1: 1.670e-11, b1: 1.414e-11, W2: 1.253e-11 and b2: 4.758e-12. All are below 1e-7; perturbed parameters are restored. This checks derivatives at the tested point, rather than proving correctness for all parameter values.

One epoch is one full-batch update on eight examples. Success requires every coordinate to be within 0.10 of its target, checked after each update; runs stop at success or 20,000 epochs. This is our evaluation rule, not an official passing threshold. We test five learning rates at s = 1 and three additional scales at rate 3, with seeds 0-9 per configuration: 80 runs total. The shared baseline is included once; failures remain in the data.

Rate	Scale s	Success	Median epochs*	Median sec.**	Median loss
0.1	1	0/10	-	1.033	0.163174
1	1	10/10	10,880.0	0.579	0.006096
3	1	10/10	3,552.5	0.179	0.006057
10	1	10/10	1,054.5	0.051	0.006188
100	1	0/10	-	1.020	0.468749
3	0	0/10	-	0.982	0.016545
3	0.01	10/10	4,557.5	0.223	0.006064
3	5	10/10	3,442.5	0.168	0.006494

* Successful runs only. ** All runs, including failures at the cap. Final loss medians also include all runs. Times are recorded local wall-clock measurements and vary with hardware and load. Scale 0 repeats one identical deterministic initialization despite different seed labels.

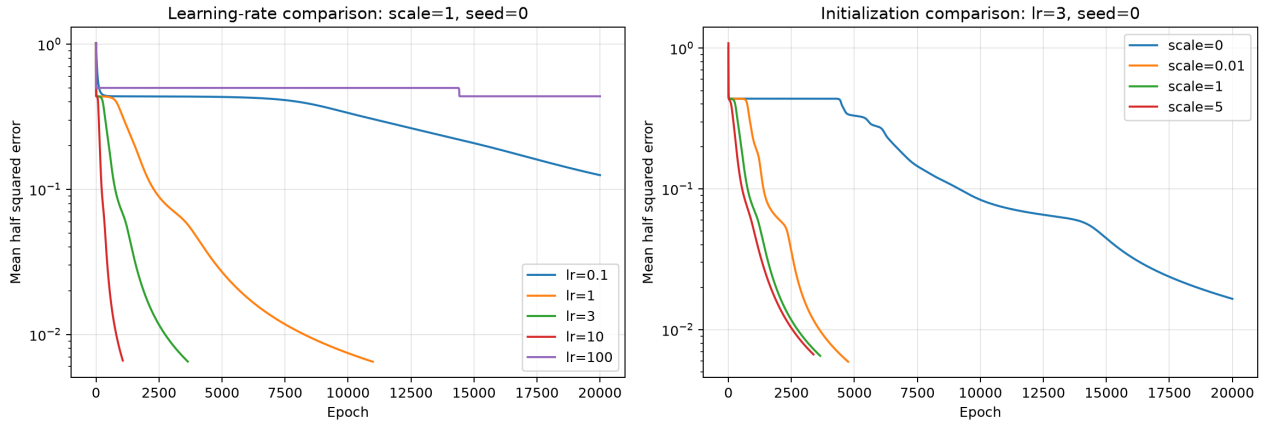


Figure 1. Seed-0 learning curves: learning rate varies at scale 1 (left); scale varies at rate 3 (right). Loss uses a logarithmic axis. Successful curves end early; the table covers all ten seeds.

Rates 1, 3 and 10 succeed in 10/10 runs. Rate 10 is fastest among tested rates, with median 1,054.5 epochs versus 3,552.5 at rate 3 and 10,880 at rate 1. Rate 0.1 learns too slowly for this budget; rate 100 also fails. This agrees with the lecture's warning that too-small steps are slow and too-large steps can prevent convergence [2, p. 30]. Failure within the budget does not prove eventual impossibility.

At rate 3, scales 0.01, 1 and 5 all succeed; their median epochs are 4,557.5, 3,552.5 and 3,442.5. The smallest random range is slower here; scale 5 is slightly faster than scale 1. Zero initialization fails the criterion despite reducing loss. Identical hidden units preserve symmetry in exact arithmetic, although finite-precision perturbations can break ideal symmetry over long runs. Ten tested seeds provide sampled evidence of reliability, not a universal guarantee.

4. Reconstruction and the hidden representation

The baseline uses seed 0, rate 3 and scale 1. It succeeds after **3,644 epochs**, or 29,152 pattern presentations. The standalone notebook run takes 0.193 seconds. Loss decreases from 1.016208 to 0.006506; maximum absolute error is 0.099990. All 8/8 complete output vectors match their targets after thresholding at 0.5.

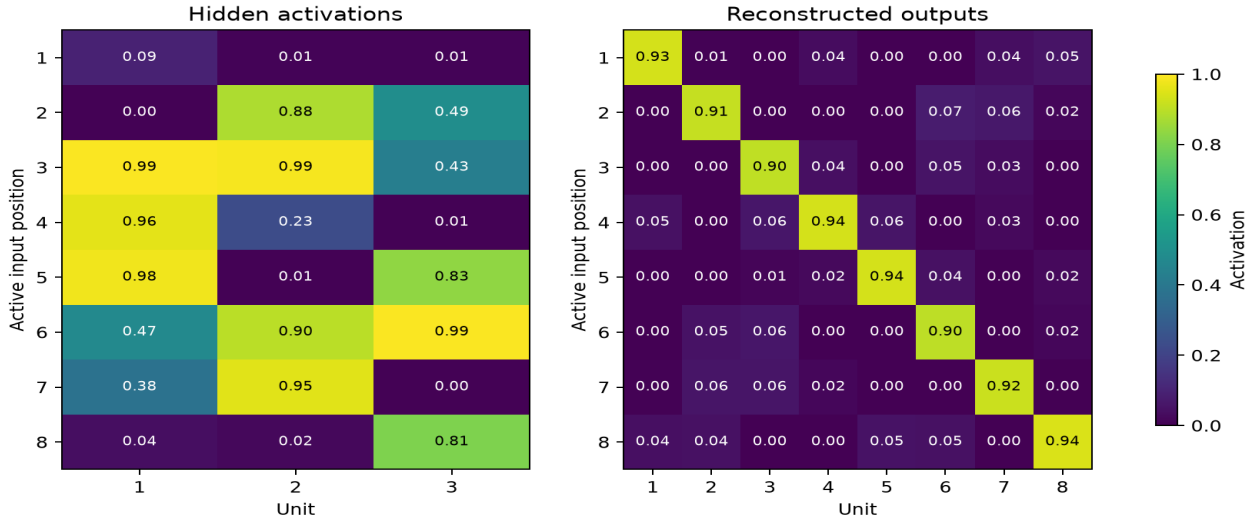


Figure 2. Baseline hidden activations and reconstructions. Rows identify the active input position. The output diagonal is high and off-diagonal entries are low; displayed zeros are rounded values.

Input	Hidden H1	Hidden H2	Hidden H3	Code at 0.5
1	0.094	0.008	0.011	000
2	0.005	0.878	0.491	010
3	0.993	0.986	0.429	110
4	0.959	0.232	0.007	100
5	0.978	0.008	0.834	101
6	0.470	0.898	0.993	011
7	0.378	0.955	0.004	010
8	0.036	0.023	0.807	001

H1 is above 0.5 for positions 3, 4 and 5; H2 for 2, 3, 6 and 7; H3 for 5, 6 and 8. Each unit detects an overlapping group of positions. Together they produce eight distinct continuous representations but only seven thresholded codes. Positions 2 and 7 both give 010, yet their activations differ: [0.005, 0.878, 0.491] versus [0.378, 0.955, 0.004]. The decoder uses these continuous values, so unique binary labels are unnecessary.

For one-hot input e_i, hidden preactivation j is W1[i,j] + b1[j]. For input 1 and H1 this is -2.320 + 0.054 = -2.266, giving sigmoid approximately 0.094. For output 7, the weights from H1, H2 and H3 are approximately [-2.132, 6.567, -11.176], with bias -2.978. The negative H3 weight contributes to suppressing output 7 for input 2, whose H3 activation is much larger than input 7's. All three weighted activations and the bias jointly determine the output; a weight sign alone is not a complete explanation.

5. Scope and references

These results demonstrate reconstruction of all eight training patterns, not generalization to unseen mixtures or noisy inputs. The learned code depends on initialization; sigmoid outputs approximate binary targets rather than being mathematically exact zeros and ones.

[1] Lab - NN.pdf, pp. 1-2: architecture, task and deliverables. [2] 1.1-Linear models.pdf, pp. 26-27, 30: gradient descent and learning rate. [3] 1.2-ANN.pdf, pp. 10, 15-21, 58: forward propagation, derivatives, updates and small random weights. [4] Backprop Help.pdf, section 2.3: supplementary numerical gradient checking.